

MARAN SUITE SYSTEM — Prompts para Replit

Usar en orden estricto. No pasar al siguiente hasta que el anterior esté completo y sin errores.

PROMPT 1 — Estandarizar tipos de IDs

Pegar esto en Replit tal cual:

Necesito estandarizar los tipos de IDs en `shared/schema.ts`. El problema es que la mayoría de las tablas usan `varchar` con UUID como primary key, pero 6 tablas nuevas usan `serial` (integer). Hay que convertir esas 6 tablas a UUID para que todo sea consistente antes de migrar a PostgreSQL.

Las 6 tablas que hay que cambiar de `serial` a `varchar` con UUID son:

- `bedTypes`
- `systemNotifications`
- `webCheckins`
- `guestPreferences`
- `stayNotes`
- `hospitalityAlerts`

Para cada una de estas tablas hacer los siguientes cambios en `shared/schema.ts`:

1. Cambiar `id: serial("id").primaryKey()` por `id: varchar("id").primaryKey().default(sql\`gen_random_uuid()\`)`
2. En todas las tablas que referencian estas tablas como foreign key, cambiar los campos de referencia de `integer` a `varchar`. Por ejemplo si hay un campo `preferenceId: integer(...)` que referencia `guestPreferences`, cambiarlo a `preferenceId: varchar(...)`.
3. Verificar que en `server/storage.ts` todos los métodos que crean registros de estas

tablas generen un UUID usando `randomUUID()` del módulo `crypto` (ya importado en `routes.ts`) en lugar de depender de un autoincrement. El patrón correcto es:

```
const newRecord = { id: randomUUID(), ...data }
```

4. Después de los cambios, verificar que el proyecto compile sin errores TypeScript ejecutando el check de tipos.

No tocar ninguna otra tabla. Solo las 6 mencionadas.

PROMPT 2 — Corregir campos de fecha

Pegar esto en Replit después de que el Prompt 1 esté completo:

En `shared/schema.ts` hay varios campos de fecha que están definidos como `text` en lugar del tipo correcto. Necesito corregirlos para que la migración a PostgreSQL funcione bien y las queries de fechas sean correctas.

Buscar en todo el archivo `shared/schema.ts` los campos que sigan estos patrones y corregirlos:

1. Campos llamados `createdAt`, `updatedAt`, `deletedAt`, `closedAt`, `completedAt`, `paidAt`, `readAt`, `resolvedAt`, `acknowledgedAt` que estén definidos como `text(...)` → cambiarlos a `timestamp("nombre_campo").defaultNow()`
2. Campos llamados `lastLogin`, `termsAcceptedAt`, `expiresAt` que estén como `text(...)` → cambiarlos a `timestamp("nombre_campo")`
3. Campos que sean fechas puras (sin hora) como `checkIn`, `checkOut`, `startDate`, `endDate`, `birthDate`, `validFrom`, `validUntil` que estén como `text(...)` → cambiarlos a `date("nombre_campo")`

Importante: NO cambiar campos que intencionalmente guardan texto libre, como `notes`, `description`, `name`, etc. Solo cambiar los campos que claramente son fechas por su nombre.

Después del cambio, verificar que compile sin errores TypeScript.

PROMPT 3 — Migración a PostgreSQL

Pegar esto en Replit después de que los Prompts 1 y 2 estén completos y sin errores:

Es momento de migrar el sistema de MemStorage a PostgreSQL real. El proyecto ya tiene Drizzle ORM configurado y `drizzle.config.ts` apunta a `DATABASE_URL`. Necesito hacer la migración completa.

Seguir estos pasos en orden:

Paso 1 — Verificar la base de datos Confirmar que la variable de entorno

`DATABASE_URL` está configurada en el proyecto de Replit. Si no está, crearla apuntando a la base de datos PostgreSQL de Replit.

Paso 2 — Crear el archivo de conexión a la base de datos Crear `server/db.ts` con el siguiente contenido:

```
import { drizzle } from "drizzle-orm/node-postgres";
import { Pool } from "pg";
import * as schema from "@shared/schema";

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: process.env.NODE_ENV === "production" ? { rejectUnauthorized: false } : false
});

export const db = drizzle(pool, { schema });
```

Paso 3 — Ejecutar la migración del schema Ejecutar en la terminal: `npm run db:push` Esto crea todas las tablas en PostgreSQL según el schema de Drizzle. Confirmar que no hay errores.

Paso 4 — Crear el nuevo storage con PostgreSQL Crear un nuevo archivo `server/db-storage.ts` que implemente la misma interfaz que `server/storage.ts` pero usando queries reales de Drizzle en lugar de arrays en memoria.

Empezar por los métodos más críticos en este orden:

1. Todos los métodos de `rooms` y `roomTypes`
2. Todos los métodos de `guests` y `companies`
3. Todos los métodos de `reservations`, `charges` y `payments`
4. Todos los métodos de `housekeepingTasks`

5. El resto de los módulos

Para cada método, el patrón de Drizzle es:

```
// GET todos
async getRooms(): Promise<Room[]> {
  return await db.select().from(schema.rooms);
}

// GET por ID
async getRoom(id: string): Promise<Room | undefined> {
  const result = await db.select().from(schema.rooms).where(eq(schema.rooms.id, id));
  return result[0];
}

// CREATE
async createRoom(room: InsertRoom): Promise<Room> {
  const result = await db.insert(schema.rooms).values({ id: randomUUID(), ...room });
  return result[0];
}

// UPDATE
async updateRoom(id: string, room: Partial<InsertRoom>): Promise<Room | undefined> {
  const result = await db.update(schema.rooms).set(room).where(eq(schema.rooms.id, id));
  return result[0];
}

// DELETE
async deleteRoom(id: string): Promise<boolean> {
  const result = await db.delete(schema.rooms).where(eq(schema.rooms.id, id)).returning();
  return result.length > 0;
}
```

Paso 5 — Migrar los datos seed Una vez que `db-storage.ts` tenga los métodos de inserción funcionando, crear un script `server/seed.ts` que inserte todos los datos de ejemplo que hoy están en `server/storage.ts` (habitaciones, huéspedes, tipos de habitación, menú del restaurante, etc.) directamente en PostgreSQL.

Paso 6 — Cambiar el import en routes.ts Cuando `db-storage.ts` esté completo, cambiar en `server/routes.ts`:

```
// Antes:
import { storage } from "../storage";

// Después:
import { storage } from "../db-storage";
```

Paso 7 — Verificar funcionamiento Probar que los endpoints principales responden correctamente:

- GET /api/rooms
- GET /api/reservations
- GET /api/dashboard/stats

Confirmar que los datos persisten después de reiniciar el servidor.

PROMPT 4 — Autenticación y roles

Pegar esto en Replit después de que el Prompt 3 esté completo y el sistema funcione con PostgreSQL:

Necesito implementar autenticación completa en el sistema. Las dependencias ya están instaladas (`passport` , `passport-local` , `express-session` , `connect-pg-simple`). La tabla `systemUsers` ya existe en el schema con campos `username` , `password` , `role` , `isActive` .

Implementar los siguientes cambios:

1. Configurar sesiones y passport en `server/index.ts`

Agregar después de los middlewares existentes de `express.json`:

```
import session from "express-session";
import connectPgSimple from "connect-pg-simple";
import passport from "passport";
import { Strategy as LocalStrategy } from "passport-local";

const PgSession = connectPgSimple(session);

app.use(session({
  store: new PgSession({
    connectionString: process.env.DATABASE_URL,
    tableName: "sessions",
    createTableIfMissing: true,
  }),
  secret: process.env.SESSION_SECRET || "maran-suite-secret-key",
  resave: false,
  saveUninitialized: false,
  cookie: {
```

```

    secure: process.env.NODE_ENV === "production",
    maxAge: 8 * 60 * 60 * 1000, // 8 horas (un turno de trabajo)
  },
}));

app.use(passport.initialize());
app.use(passport.session());

```

2. Configurar la estrategia de login

Crear `server/auth.ts` con:

- Estrategia local que busca el usuario por `username` en `systemUsers`
- Verificación de password (usar `bcrypt` para hashear — instalar con `npm install bcrypt @types/bcrypt`)
- Serialización/deserialización de sesión
- Función `hashPassword(password)` y `verifyPassword(password, hash)`

3. Agregar endpoints de autenticación en `server/routes.ts`

```

POST /api/auth/login    → Login con username y password
POST /api/auth/logout   → Cerrar sesión
GET  /api/auth/me       → Usuario actual (para que el frontend sepa quién está lo

```

4. Middleware de autenticación

Crear función `requireAuth` que verifica si hay sesión activa. Si no hay sesión, retornar 401.

Aplicar `requireAuth` a TODAS las rutas `/api/` EXCEPTO:

- `POST /api/auth/login`
- `GET /api/public/web-checkin/:token`
- `POST /api/public/web-checkin/:token`
- `POST /api/webhook/chatbot`

5. Middleware de roles

Crear función `requireRole(roles: string[])` que verifica que el usuario tiene uno de los roles permitidos.

Aplicar a las rutas más sensibles:

- Rutas de administración (`/api/system-users` , `/api/system-settings`) → solo `admin`
- Rutas de eliminación en módulos principales → `admin` o `manager`

6. Pantalla de login en el frontend

Crear `client/src/pages/login.tsx` con:

- Formulario simple con usuario y contraseña
- Logo/nombre del hotel
- Manejo de error si las credenciales son incorrectas

En `client/src/App.tsx` :

- Agregar lógica para verificar sesión al cargar la app (`GET /api/auth/me`)
- Si no hay sesión activa, mostrar la pantalla de login en lugar del sistema
- Si hay sesión, mostrar el sistema normal

7. Usuario administrador inicial

Crear un script o endpoint temporal `POST /api/auth/setup` que solo funcione si no hay ningún usuario en el sistema, que cree el primer usuario admin con:

- `username:` `admin`
- `password:` `maran2026` (hasheado con `bcrypt`)
- `role:` `admin`

Este endpoint debe deshabilitarse automáticamente una vez que existe al menos un usuario.

Fin de los prompts de infraestructura base

Una vez completados estos 4 prompts, el sistema está listo para uso real en el hotel.